

Modelado con casos de Uso y UML: Del Negocio al Sistema

Nombre del autor

 **Tiempo estimado: 10 min**

Universidad San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ingeniería en Ciencias y Sistemas.
Segundo Semestre 2026
[Laboratorio - Nombre del Curso - Sección]

1. Marco Formativo

1.1 Valores

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Paciencia Reflexiva	Modelar antes de codificar permite detectar inconsistencias tempranas, ahorrando horas de reprogramación
Comunicación Clara	Usamos lenguaje natural en CUN/CUS para alinear la visión de stakeholders no técnicos con el equipo de desarrollo
Rigurosidad Técnica	Aplicamos estándares UML y plantillas estructuradas para garantizar trazabilidad y calidad en la especificación de requisitos

1.2 Competencia(s)

Tipo de Competencia	
Competencia General	Modelar sistemas de software utilizando Casos de Uso del Negocio (CUN) y el Lenguaje de Modelado Unificado (UML), identificando actores, relaciones y niveles de abstracción, con el fin de especificar requisitos funcionales de manera clara, estructurada y documentada
Competencia Específica	Diferencia entre CUN y CUS, aplica correctamente las relaciones «include» y «extend», y elabora diagramas de clases y de secuencia para representar la estructura y comportamiento dinámico del sistema

2. Palabras Clave

CUN vs CUS: Los Casos de Uso del Negocio describen procesos completos desde la perspectiva del rol organizacional; los del Sistema detallan funcionalidades específicas del software.

Actor: Rol (no persona específica) que interactúa con el sistema o proceso de negocio.

Core del Negocio: Óvalo inicial que delimita el alcance total a modelar, validado con stakeholders.

Descomposición y Expansión: Proceso de dividir el core en procesos de negocio y luego detallar flujos, variaciones y pasos exactos.

«**include**»: Relación obligatoria donde un caso de uso reutiliza comportamiento repetitivo de otro.

«**extend**»: Relación opcional que añade comportamiento bajo condiciones específicas sin alterar el flujo base.

Diagrama de Clases: Representa la estructura estática (entidades, atributos, relaciones).

Diagrama de Secuencia: Modela la interacción temporal entre objetos para cumplir un caso de uso.

3. Introducción

El modelado de sistemas es el puente entre las necesidades del negocio y la implementación técnica. Antes de escribir una sola línea de código, es fundamental capturar **qué** debe hacer el sistema mediante Casos de Uso y representar su estructura y comportamiento con UML. Esta lectura complementaria explora cómo transitar desde el Core del Negocio hasta la especificación técnica, utilizando relaciones estándar («**include**», «**extend**») y diagramas clave (Clases y Secuencia) para reducir ambigüedades, prevenir errores costosos y entregar software alineado con la realidad operativa.

4. Contenidos

Del Proceso de Negocio a la Especificación Técnica

Diferenciación y Evolución de Casos de Uso

Tabla 1 - Comparativa CUN vs CUS

Campo	Casos de Uso del Negocio (CUN)	Casos de Uso del Sistema (CUS)
Nivel de abstracción	Proceso de negocio completo	Funcionalidad específica del software
Actor	Rol organizacional (cliente, gerente, proveedor)	Usuario final, sistema externo, interfaz
Notación UML	Óvalo con línea diagonal (/)	Óvalo simple
Objetivo	Entender el flujo de valor organizacional	Especificar cómo el software responde a estímulos

Subtema: Cuándo usar «include» vs «extend»

Situación	Relación	Justificación
Verificar stock al procesar pedido	«include»	Ocurre siempre, es parte obligatoria del flujo
Enviar notificación por correo	«extend»	Solo se ejecuta si el usuario activa la opción
Validar formato de archivo al subir	«include»	Condición técnica inherente, sin ella falla el caso base

Exportar reporte a PDF	«extend»	Comportamiento adicional opcional, el reporte ya es válido sin él
------------------------	----------	---

Caso de estudio

Sistema: Gestión de Reservas para Clínica Médica

Contexto: La clínica necesita digitalizar la agenda de citas. El proceso inicia cuando el paciente solicita una cita, se verifica disponibilidad, se confirma y se envía recordatorio.

Aplicación del Modelado:

1. **Core:** Gestionar Citas Médicas (actores: Paciente, Recepcionista, Médico)
2. **Descomposición:**
 - Solicitar Cita
 - Confirmar Disponibilidad
 - Enviar Recordatorio
3. **Expansión y Relaciones:**
 - Solicitar Cita «include» Verificar Disponibilidad de Horario (obligatorio)
 - Solicitar Cita «extend» Aplicar Descuento por Fidelidad (solo si el paciente cumple condiciones)
 - Confirmar Cita generaliza a Confirmar Cita Presencial y Confirmar Cita Telemedicina
4. **Plantilla Textual (Extracto):**
 - **Propósito:** Registrar y validar una cita médica en el sistema.
 - **Secuencia Normal:** 1. Paciente selecciona especialidad → 2. Sistema muestra horarios → 3. Paciente elige → 4. Sistema valida → 5. Se genera confirmación.
 - **Excepción:** Paso 4: Si no hay disponibilidad, el sistema sugiere alternativas o lista de espera.
5. **UML:**
 - **Clases:** Paciente, Cita, Médico, Horario (relaciones 1:N, 1:1)
 - **Secuencia:** Paciente → Interfaz → ControladorCita → BaseDatosHorarios → Confirmación

Resultado: El modelado previo evitó que el equipo codificara lógica de descuentos en el flujo principal, separó responsabilidades técnicas y permitió que el área médica validara el flujo en lenguaje natural antes del desarrollo.

5. Conclusiones

Modelar con Casos de Uso y UML no es una etapa burocrática, sino una inversión estratégica que reduce rework, alinea expectativas y estandariza la comunicación entre negocio y desarrollo. Dominar la diferencia entre CUN y CUS, aplicar correctamente «include» y «extend», y traducir flujos a diagramas de clases y secuencia permite construir sistemas más robustos, mantenibles y centrados en el valor real del usuario. La paciencia en el modelado se traduce en velocidad en la entrega.

6. Referencias

1. **Object Management Group (OMG).** (2017). *Unified Modeling Language (UML) Version 2.5.1*. <https://www.omg.org/spec/UML/2.5.1/>
2. **Fowler, M.** (2004). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3rd ed.). Addison-Wesley.
3. **Jacobson, I., Christerson, M., Jonsson, P., & Övergaard, G.** (1992). *Object-Oriented Software Engineering: A Use Case Driven Approach*. Addison-Wesley.
4. **Larman, C.** (2004). *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development* (3rd ed.). Prentice Hall.
5. **Cockburn, A.** (2000). *Writing Effective Use Cases*. Addison-Wesley